

Assessment Task 2

FormBridge Restaurant Enquiry Workspace

Group project technical report

36118 Applied Natural Language Processing

Group member	Student ID
Karthik Ramesh	26043455
Avanish Rao	
Pranav Chhetri	
Shivom Karnad	25977543
Rishi Mundra	

Submitted by: Karthik Ramesh

Abstract

Our team has developed a conversational agent we call FormBridge to solve the problem of converting dynamic customer conversations with restaurant staff into booking briefs. The application tracks five booking fields, linking them to their originating messages and asking the user to provide clarification where a request is lacking. The staff reviews the amendments, sends an enquiry to their colleagues and shares a private review link with the guest. The voice input and suggestions for guest care support the process. The external messaging and reservation connections were simulated. We performed a comparison of rules, extracted the direct model and applied FormBridge to 20 previously examined development dialogues from the Schema-Guided Dialogue dataset to evaluate the accuracy of our application. Allowing for equivalent date and time formats, the methods matched 35/100, 89/100 and 97/100 fields respectively. FormBridge achieved full five-field matches in 17/20 dialogues, taking approximately double the time of direct extraction to process each conversation. An independent regression check then matched 100/100 fields on 20 previously examined test dialogues. Neither our measure nor the direct extraction is representative of the performance of approaches on unexamined enquiries.

A controlled comparison highlighted an issue with the current approach: reducing correct fields by three from 88/100 to 85/100 by applying validation to the same model responses. Twenty constructed cases passed the checks for uncertainty, clarification and approval, and the required source messages are attached to all 93 populated fields. The automated tests and one walkthrough revealed selected procedures to be functioning. Further investigations, including a study with restaurant staff, would be necessary to confirm usability and time-saving potential.

Contents

Section	Page
1 Problem and objectives	3
2 Product and user journey	4
3 Architecture and state	5
4 NLP methods and choices	6
5 Data and preparation	7
6 Evaluation design	8
7 Results and resource use	9
8 Error analysis and development findings	10
9 Interaction design	11
10 Functional verification	12
11 Ethics and limitations	13
12 Project delivery and next steps	14
References	15
Appendices A to F	16–21

1 Problem and objectives

The restaurant enquiry problem

A customer requests for four people at seven but later amends the request to five people at sometime after eight. We know that the current guest count is known but not the exact time. Reading only the first message gives an out-dated summary and reading only the last provides no information regarding the restaurant and date. The choice to 8:30 pm would add a decision not made by the guest.

This is a dialogue state tracking problem: maintaining what a customer currently wants as a conversation progresses. TRADE, for instance, derives state values from the conversation rather than a fixed list of answers (Wu et al., 2019). The Schema-Guided Dialogue dataset provides service descriptions, conversations and annotated states to evaluate this task (Rastogi et al., 2020).

We have selected restaurant enquiries, as the consequences of a missed correction is easy to see. Front-of-house staff must understand what the current request is and relay it to a colleague without losing a change or special requirement. Guests will need a means of checking that their plans have been understood. Our design is based on these enquiry scenarios, we have not yet interviewed restaurant staff to validate the needs or demand.

Objectives and research question

We aimed to build a web application that gives staff accurate booking information and lets them check it against the conversation. The project had four objectives:

1. Maintain restaurant, city, date, time and guest count as messages change.
2. Show the messages supporting each value and ask for clarification where details remain uncertain.
3. Compare rules with contextual extraction using a reproducible evaluation.
4. Support intake, staff review and team handoff, with private records and recovery from failed updates.

Research question: How accurately does FormBridge maintain booking details compared with rules and direct model extraction, and how can those details support staff review?

Scope and completion criteria

FormBridge receives English text and short voice messages. A transcript can be up to 30, 1,000-character messages long. Optional guest-care information appear next to the five booking fields. Staff saves enquiries in their device or in a team, private, and assigns them to a colleague and requests of guest review.

A completed workflow results in staff having a brief describing the current request, unanswered questions, and review decisions. Approval indicates that the brief was reviewed; it does not reserve a table. Emails, WhatsApp, and reservation connections are simulated. Since these workflows go beyond the five fields in the dataset, we assess them separately from extraction accuracy.

2 Product and user journey

The staff begins their working session by opening an empty booking desk. It is possible to type a conversation, import a saved enquiry or review a voice transcription. Moreover, a restaurant can publish a branded guest page or embed the assistant's interface on their website. The assistant then uses information about the restaurant and answers the request by collecting all the necessary details. The contact information is only shared with the team when a guest confirms their request.

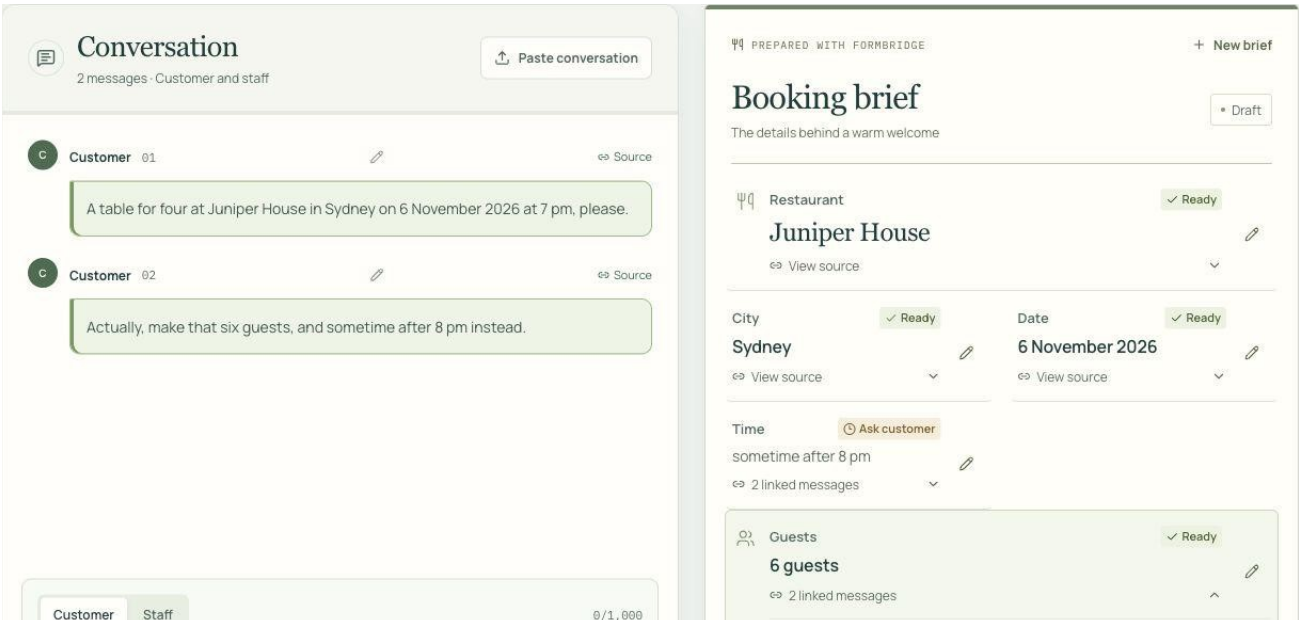


Figure 1. The booking desk after a customer changes four guests to six and replaces 7 pm with "sometime after 8 pm". The guest count updates while the exact time remains unresolved. Screenshot of the live application, 13 October 2026.

A complete enquiry

In figure 1, the restaurant, city, and date are the same. The staff member went through the conversation and history and asked about the time specifically. If the staff member specifies 8.30 pm, it will be accepted as an agreed time only after the customers confirm. The completed brief can be approved.

When the enquiry is assigned to the team, additional fields with contact details, accepted guest-care requests, an owner and due time can be added. The guest can change their enquiry by using the private review link. Their replied message is saved as a new message with the status of Needs reply, canceling any previous approvals. The simulated reservation connection lets see how the revised request is escalated to the team and how the failed attempt is returned.

Work beyond the first reply

The team desk illustrates comments, priorities, assignments, and reminders along with each enquiry. The staff members manages the returning-guest records where the merging or linking options are available to avoid duplicates, the guests may have the same names. The service planner manages the enquiries with set dates while the desk assistant prepares their responses and handoff notes for the staff's approval.

The application uses to let its team members access it through their physical devices and private team invites to facilitate collaborations. A separate simulation shows the email registration and password-recovery processes. Note that actual emails require one to configure the sender's platforms and test them.

3 Architecture and state

We used Next.js 16.3.6, React 19.3.0 and TypeScript 5.9.3 to build the application, deployed on Vercel. OpenAI SDK 7.23.0 handles language and transcription requests, while Zod 4.6.5 checks input and response formats. Supabase provides sign-in and PostgreSQL storage, with access rules applied to individual records. The dependency lockfile records the installed versions.

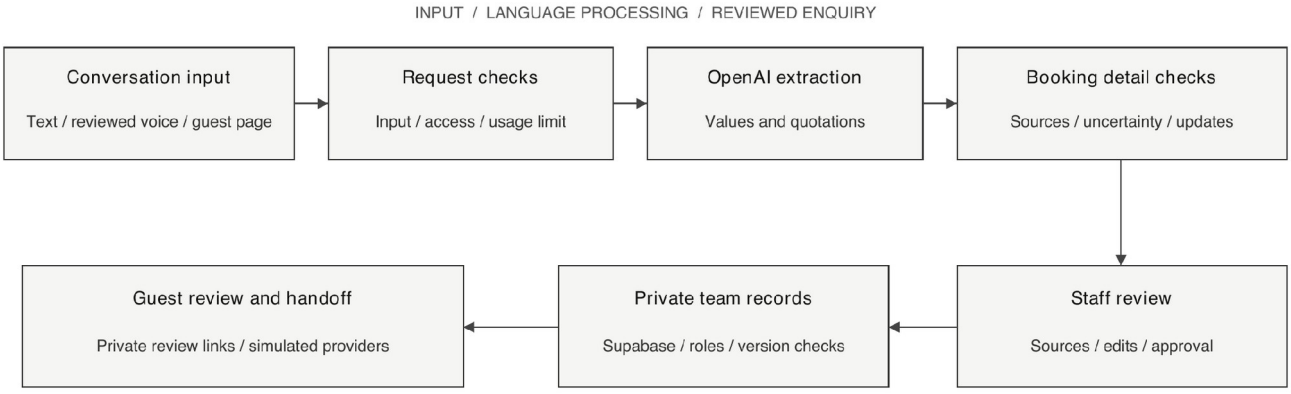


Figure 2. Application architecture. Model output is checked before it changes the brief. Shared workspace reads and changes require membership; the public booking desk works independently. External messaging and reservations use simulations.

Keeping the brief consistent

The model proposed a value for each field plus, the uncertainty and the messages justifying this proposal. It was checked by the application code against proposals before updating the brief. Each field is marked missing, resolved or waiting for clarification. The brief records if a value was taken from the model or staff edit. The approval belongs to a specific version, thus, changing the request invalidates its subsequent approval.

The active draft is stored in the browser session storage. The local library stores up to 50 saved enquiries whilst shared enquiries are stored in Supabase and are limited to team members. The guest's preferences and comments are stored separately from the five extracted fields. Imported JSON and old drafts containing model predictions should be rechecked before approval.

Handling shared edits and failed requests

Every save specifies the version being worked on. If another teammate has made changes to the version, the application warns of a conflict. It is possible to retain unrelated changes, and the application asks the user to specify which value should be preferred in the fields changed by both users. Whenever reply requests are received, they are always associated with the original enquiry, hence any delayed reply cannot appear in a different case.

Before sending any request to the OpenAI API, the server evaluates the input and subtracts an estimated amount from the shared allowance. The server then saves the reported usage. If the request failed, and no usage was reported, the estimated amount is subtracted in the allowance. If the allowance database is unavailable, AI requests will be stopped, but manual review and saved work will still be available. The application usage is limited by this allowance, which does not control all spending on the OpenAI account.

4 NLP methods and choices

Three approaches to booking state

Table 1. Approaches to booking-state extraction.

Approach	Method	Purpose
Rules	Regular expressions, number-word conversion and simple correction or acceptance patterns	An interpretable baseline without model calls
Direct extraction	The model predicts five values from the conversation so far	Tests contextual extraction with minimal processing after the model response
FormBridge	The model proposes values and quotations; code checks them before updating the brief	Adds sources, uncertainty and controlled updates for staff review

All three approaches receive the same messages with customer and staff labels. FormBridge checks whether quoted messages exist, whether their wording supports the proposed value and whether a later message changes it. It also checks customer acceptance, dates, time constraints, guest counts and cancellation. Manual edits are preserved, with later conflicts referred to staff.

Structured output gives the model response a predictable format, but its content can still be wrong (OpenAI, n.d.-b). For example, “after 7 pm” must remain a time preference, and the numeral in a time must not become a guest count. Validation checks for these errors. The checks can also reject a valid paraphrase or miss an acceptance, which makes their own effect worth evaluating.

Model and latency choices

The deployed configuration uses gpt-6-luna for the initial extraction. It permits a second call to gpt-6.1-sol to repair the response or recover from a temporary failure. Both are employing low reasoning effort, automatic retries of the SDK are disabled, and the output limits for FormBridge are 2,400 tokens and 900 for direct extraction.

A ten dialogues trial using Luna yielded a match of 45/50 fields and no failed dialogue, averaging 27.78 seconds per conversation, and a subsequent trial using Sol first and Astra for repair matched 38/50 on the same dialogues. Furthermore, there was also one failed dialogue and the nine successfully completed dialogues averaged 52.39 seconds. We have retained Luna as the practical starting model, although the prompts and validation also changed between trials, and cannot tell which model is intrinsically better. Appendix E records both configurations.

The first call has a limit of 12 seconds and the additional call has a limit of 14 seconds, leaving room for application processing within the 30-second request limit. However, the separate benefit of Sol repair has yet to be measured. Using a hosted model avoids the need for training and hosting model weights, but introduces network dependence and external data processing. The model names are aliases, and their behaviour may change over time.

Language features around the brief

Voice input uses `gpt-transcribe`, and staff review its transcript before adding it to the conversation. Guest-care suggestions include customer quotations and require individual acceptance. The desk assistant proposes notes and follow-ups for review. Answers to restaurant questions use matching categories of approved information and acknowledge when an answer is unavailable.

5 Data and preparation

Dataset and mapping

We evaluated extraction using Google's Schema-Guided Dialogue dataset (Google Research, n.d.). We retained conversations about one restaurant service that included a customer turn labelled `ReserveRestaurant`. Table 2 shows the eligible subset before applying the application's 30-message limit. Appendix B identifies the exact dataset revision.

Table 2. Eligible restaurant conversations before applying the message limit.

Split	Restaurant service	Eligible dialogues	Reservation customer turns
Train	Restaurants_1	367	1,870
Development	Restaurants_2	73	385
Test	Restaurants_2	73	342

The training schema uses `city` and `party_size`; the development and test schemas use `location` and `number_of_seats`. The loader maps both to the application's city and guest-count fields. Restaurant name, date and time complete the five-field task. No model weights were trained on these data.

Preparing each example

The loader reads in files in the order specified and terminates the conversations at a customer turn about a reservation. Then it matches USER and SYSTEM to customer and agent respectively and labels the messages with identifiers. The final annotated values of this dataset are the answers that powered this scoring. Each approach processes the conversation again at every customer message, using only those messages that were available at the given step in the conversation. These are also called “prefixes.”

Expected answers, defaults according to the service schema, and future messages are excluded as inputs to the model. The history is given without a reference date, and therefore “tomorrow” should not be considered as the day of the experiment. A date without a year is a valid value but should be flagged for clarification before approval.

Other inputs and data limitations

The evaluated application processes the real-time dialogues, taking the user’s conversations and the approved information on restaurants, including transcripts of dialogues, as inputs. The screenshots and tests were made using the fictional restaurants and guests, separately from the dataset. The labels of the SGD’s 5 fields contain no information on the guest-care or staff organization. SGD allows for the evaluation of booking values. However, the assistant labels the quotes supporting the booking, clarifying questions, and staff workload. The tool is open-sourced; thus, we cannot confirm whether it was trained on the hosted model. The conversations were tested and reviewed for development; by allowing retesting, we can catch regression. However, we cannot assess the performance on the unseen data.

The upstream dataset is distributed under the Creative Commons Attribution-ShareAlike 4.0 International License. All excerpts and redacted versions of the dataset should include the proper licensing and attribution for the copyright holders. The project has the corpus version and license for the evaluation materials.

6 Evaluation design

Comparing the three systems

We ran the released extraction code on two previously examined samples: 20 development dialogues for all three methods and 20 test dialogues for FormBridge regression testing. The dialogue selections were saved before making API calls. Appendices A and B provide the code version, configuration and selected conversations.

All methods see identical ordered conversation prefixes and finish at the same customer turn. Direct extraction and FormBridge use GPT-6 Luna as the primary model. FormBridge also permits a Sol repair or recovery and requests quotations. This compares complete systems, with different prompts, output allowances and call counts.

Controlled component and feature checks

To evaluate the influence of the validation process, an additional model response was requested by the end of each development conversation. The same response was scored both before and after our checks. The prompt, model, allowance for the output and input all remained the same in the two cases. This means that the only differing factor is the presence of the validation checks, which is isolated from the other elements of the application. One of the requests has timed out and thus counts as a failure both times.

We also applied 20 constructed conversations to test repair calls, customer acceptance, cancellations, dates, the number of guests as well as expressions of uncertainty. The expected values, statuses, supporting information and clarification targets were recorded in the calls, but hidden from the model at the time of processing. Each transcript was evaluated from an empty brief. The tests ensure that the required sources and clarifications are present, as well as that the approval has been reached at the right point.

Metrics

Field accuracy is the ratio of the number of correct final values to the 5× number of attempted dialogues, giving 20 dialogues the denominator of 100. Complete-brief accuracy is the ratio of dialogues that had all 5 fields matched to attempted dialogues, counting failed dialogues in the denominator. Staff edits are excluded.

The strict matcher is using the text normalization and the canonical matcher additionally accepts equivalent date/time formats while preserving the missing year and distinction of range and specific time, both are reported. The numbers reflect the degree of match across the 5 fields that we selected and do not comment on the success of the overall SGD task, or readiness of the brief for approval.

Resource accounting encompasses OpenAI’s requests, observed tokens, and whole-dialogue replay time (replays have multiple prefixes so it’s not per message latency) and failed requests may not be billed, and token counts may not be directly reported.

Reproduction and interpretation

The evaluation calls the application's extraction functions with real OpenAI requests. Separate browser checks test the deployed interface. Saved results include predictions, failures and settings, with code checksums to identify the tested version. Appendices B and E explain how to repeat the experiments and which files are checked for changes. Hosted models can return different answers on a later run even when the code is unchanged.

7 Results and resource use

Development comparison

All three methods completed the 20 development dialogues on 13 October 2026. Table 3 reports 100 field checks per method; a complete brief requires all five values to match.

Table 3. Development accuracy on 20 dialogues.

Method	Strict fields	Canonical fields	Strict complete	Canonical complete
Rules	35/100	35/100	0/20	0/20
Direct Luna	83/100	89/100	6/20	11/20
FormBridge	97/100	97/100	17/20	17/20

FormBridge matched eight more fields and six more complete briefs than direct extraction under canonical scoring. Both found every restaurant and city; dates and guest counts accounted for most differences. The result includes differences in prompts, response formats and additional calls. Validation is examined separately below.

Effect of validation on identical proposals

Table 4. The same model responses scored before and after validation.

Condition	Canonical fields	Complete briefs
Original model proposal	88/100	14/20
Same proposal after validation	85/100	12/20

One request timed out, counting as five wrong fields in each condition. Validation gained one match but rejected four correct values, reducing accuracy. Section 8 examines these losses.

Processing time and API usage

Each method processed 166 partial conversations. Direct extraction made 166 OpenAI requests, using 93,947 input and 21,817 output tokens. FormBridge made 196 requests and recorded 381,706 input and 61,309 output tokens. Three dialogues recovered after failed requests without usage information, so FormBridge's recorded tokens understate the total.

Processing a whole dialogue averaged 21.87 seconds for direct extraction and 45.11 seconds for FormBridge, approximately twice as long. These measurements cover all steps, not one message. Concurrent evaluation activity also limits the timing comparison. Rules required no API calls.

Regression results

The previously examined test selection matched 100/100 fields and 20/20 complete briefs under both matchers. FormBridge made 149 calls across 137 partial conversations, recording 289,466 input and 50,732 output tokens. Processing averaged 32.93 seconds per dialogue, with no failures or missing usage records.

8 Error analysis and development findings

The three remaining field errors

Comparing the incorrect predictions with their conversations revealed two cases of missed acceptance across earlier messages. The third was a wording difference that the scoring code did not recognise as equivalent.

Table 5. FormBridge development mismatches.

Dialogue / field	Observed output	Interpretation
1_00005 / date	Missing; label is “today”	Staff proposed today before venue corrections. The later accepted summary omitted the date; the earlier proposal was not retained.
1_00017 / guests	Missing; label is 2	Staff proposed two guests before date and venue changes. The accepted final summary omitted the count; it was not retained.
4_00069 / date	“the 5th of this month”; label is “5th of this month”	The date is retained as uncertain. The matcher does not remove the leading article from this relative phrase.

The last case expresses the expected date. A revised matcher should recognise this equivalence and be applied to every method alongside the existing score.

The first two cases count as errors against the dataset labels. These runs saved final states without the original proposals, so they cannot establish whether extraction omitted the values or validation rejected them.

Why validation rejected correct values

The controlled experiment retained the original responses. Validation rejected San Francisco in 4_00080 because the customer said “San Fran”. In 4_00089, quotations omitted earlier customer support for two correct values. In 1_00005, the date cited a staff summary without the required acceptance. These account for all four losses: the information was present, but the checks could not establish the necessary link.

The gained match moved “Wednesday next week” from a constraint into the value field while retaining uncertainty. Six date mismatches in the full direct-model comparison similarly kept the phrase only as a constraint. A value score can overlook information preserved elsewhere, which is why we test uncertainty and approval separately.

Development lessons

The baseline rule did not match against any dates as its patterns recognise full dates, where as this sample uses relative or yearless dates. A more comprehensive date parser and a trained slot tagger would be required in order to make the comparison for FormBridge to be able to recover the missing sources and recognise valid aliases without accepting unsupported values.

In previous testing negated cancelations were being misread, rejected corrections left old approvals active and dismissed errors removed the retry control. Improvements to cancellation, approval and recovery handling dealt with these cases, and linking of delayed replies to their original enquiry fixed replies being displayed in the wrong case. Targeted tests and deployed checks have verified these corrections.

9 Interaction design

We attached the conversation to the booking card as a way for staff who need to review the detail along with the conversation to open it without moving between the two. Each field is opened to show the source messages on demand. We enabled the staff to edit the message, change the speaker, or view the previous value. We attached unresolved detail to questions and cancelled approvals when the request was changed. We followed the guidance of explaining the changes and enabling corrections. (Amershi et al., 2019).



Figure 3. Enlarged source panel from Figure 1. Staff can read the original request and correction beside the six-guest value. The separate time question preserves the customer's “after 8 pm” preference.

Eight tasks completed on the live application

We performed a walkthrough of the process that involved eight items from desks opening to an empty desk through to editing the approve enquiry. In each of the eight items, the expected outcome occurred. The history showed the four guests turned to six and left the time at 7 pm unresolved. The offer for 8:30 left the approval button inactive until the customer had accepted the change. The edit to the number of guests changed the summary and cancelled the approval. Appendix F records our walkthrough.

The cream background, the dark green panels for the navigation, and the serif font heading produce the illusion of a restaurant. The labels explain what the status means and provide additional information beyond the colour coding. The source panels are placed so that the staff can review and edit the information close to where it appears. Our walkthrough makes visible results that show the intended interaction occurs within the context of the scenario. However, the test does not involve real users in the walkthrough, and we cannot evaluate the learning curves or if they could perform the discovered actions.

10 Functional verification

The released application passed 230 automated tests across 27 files, TypeScript checking and the production build. Additional checks on the deployed application covered team permissions, simultaneous edits and complete workflows. Appendix C links the results to the code versions tested.

Table 6. Verification scope and results.

Evidence	Result and scope
Automated tests	230 passed: state, evidence, dates, input limits, request scoping and email-preview transitions
Production API/database	31/31 checks passed with real Supabase identities, permissions, shared records and routes
Restaurant browser workflows	22 recorded workflows: intake, consent, care, review, team work, widget and simulated handoff
Email preview	Local and deployed account flows completed; no real mail or production account created

The email-preview release left the tested restaurant extraction and shared-workspace code unchanged. The counts above are kept separate because some checks cover the same behaviour.

Source, uncertainty and clarification checks

All 20 constructed conversations returned the expected field values and statuses, giving 100/100 correct checks for each. They required 20 OpenAI requests, with no final failures. Table 7 examines the features that field accuracy alone does not measure.

Table 7. Feature checks on constructed conversations.

Check	Matched	Meaning
Required source messages	93/93 populated fields	Each correct value included the supporting messages specified before testing
Uncertainty retained	7/7 uncertain fields	Expected status, value and constraint were preserved
Clarification target	9/9 required questions	Each question addressed the expected unresolved detail
Ask or remain silent	20/20 cases	Missing details triggered questions; complete or cancelled requests did not
Approval eligibility	20/20 cases	Approval matched the expected complete, incomplete or cancelled state

The source check ensured that the expected messages are in place; it does not evaluate each of the other quote individually. The clarification check discovers what level of detail the question is concerning. The valuation of its phrasing would require the users' feedback. The checks of the deployed application denied another team's access, identified obsolete edits and retained inquiries after simulating provider failures. The chosen desktop and mobile layouts matched their viewports without necessitating a horizontal scroll. The keyboard checks comprised of closing the dialogues and bringing back the focus. The broader browser and assistive-technology testing remains to be done.

11 Ethics and limitations

Guest information and access

Consider that conversations may contain names, contact details, and dietary requirements. The guest page asks that you agree to share your contact details with the restaurant. Records kept by private team members are forbidden by PostgreSQL row level security (prevent reads) and checks to the staff member's permission and record version (Supabase, n.d.). Links to reviews expire after 72 hours and only expose the details required for a review. Anyone that has a link has access to it; you should treat any links shared as private.

OpenAI takes the submitted text or audio required to fulfill a request. Responses API calls have store: false, which does not guarantee that by default no provider stores the information; applicable abuse-monitoring and account controls still matter (OpenAI, n.d.-a). FormBridge does not retain raw voice recordings. JSON exports include transcript data and should be treated as guest records.

Enquiries that are shared do not currently have an automatic deletion schedule or complete procedure for handling their requests to access or delete personal information. Before using them in a restaurant, you should set up procedures and explain them to guests, including how their external services process their information and how service-plan limits and outages can impact the availability of services.

Human decisions and consequences

An allergy or accessibility request could be mistakenly extracted. The staff would still have to ask the guest and the restaurant to make sure that appropriate arrangements were made. The interface lets staff reject suggestions and check their sources. Also, there needs to be an explicit decision to link returning-guest records, thus reducing the chance of different people with the same name getting accidentally merged.

Prompts and validation should prevent the model from being steered into different work by instructions built into a conversation, but prompt injection could still happen. Shared usage limits the amount of usage on the API, and a failed update will hold the last valid brief. Testing can only address the failures and malevolent inputs that were considered.

Evaluation and deployment limits

The evaluation includes English restaurant inquiries and small samples already tested during development. The model's aliases may vary, and we cannot exclude the possibility that the public dataset was part of the model's training. As a result, these results support the claims about the cases tested, rather than overall accuracy. Independent data and independent assessments of the quality of quotation and observation of restaurant staff would improve the evidence.

Guest requests use the Australia/Sydney timezone to interpret dates, while the reminders follow the staff device. Wider usage would require a timezone setting for each restaurant. Reminders update only when the application is open. The device sign-in is tied to the browser and currently offers no email recovery.

Messaging, reservation transfer and email account recovery are still simulations. Real email delivery requires a configured sender. FormBridge does not check live restaurant availability, make reservations or process payments.

12 Project delivery and next steps

Project phases and responsibilities

We began by defining the restaurant enquiry problem and five booking fields. We then implemented and compared the NLP methods, investigated errors and expanded the interface into a shared enquiry workflow. Deployment checks, setup instructions and product videos completed the delivery phase.

All five group members contributed to building FormBridge. Karthik Ramesh (26043455) took the technical lead, while application development was shared across the group.

Table 8. Group contributions to the application.

Group member	Contribution
Karthik Ramesh	Technical lead: application development, NLP integration, evaluation tooling, deployment and technical verification
Avanish Rao	Shared application development
Pranav Chhetri	Shared application development
Shivom Karnad	Shared application development
Rishi Mundra	Shared application development

Recommendations in priority order

1. **Test with restaurant staff.** Compare similar tasks with and without FormBridge, vary their order and measure completion time, errors and reasons for corrections. Use fictional guest data and obtain consent.
2. **Improve source checks.** Address the aliases and missing citations found in the controlled experiment. Set aside independent data before further tuning and use a second annotator to assess sources and clarification. Strengthen the rule baseline and measure repair separately.
3. **Prepare for restaurant use.** Add recoverable staff accounts, restaurant timezones, retention controls and background reminders. Verify email delivery and replace simulated connectors only after provider authorisation and testing.
4. **Reduce waiting and cost.** Test whether fewer repair calls or shorter evidence responses can preserve accuracy while reducing processing time.

Conclusion

We have developed FromBridge to keep up a restaurant inquiry comprehensible to the evolving ideas of a customer. It enables staff to inspect the messages that make up every piece, resolve missing information and pass on a revised brief to a colleague. From the development comparison, FormBridge matched 97 of every 100 fields and all five fields in 17 of 20 dialogues, outperforming direct extraction at the expense of more API usage and processing time.

The controlled comparison unveiled the areas where the approach falls short: strict testing of sources can inadvertently deprive correct values. Improving source checks, evaluating unfamiliar inquiries and observing restaurant staff are the next steps towards establishing whether FormBridge improves a routine inquiry handling.

References

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human–AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (pp. 1–13). Association for Computing Machinery. https://doi.org/10.1145/3290605.3300233

Google Research. (n.d.). *Schema-Guided Dialogue dataset* [Data set; revision e852981ae34990f435897962585425930 2feaa78]. GitHub. https://github.com/google-research-datasets/dstc8-schema-guided-dialogue/tree/e852981ae34990f435897962585425930 2feaa78

OpenAI. (n.d.-a). *Data controls in the OpenAI platform*. Retrieved October 13, 2026, from https://developers.openai.com/api/docs/guides/your-data

OpenAI. (n.d.-b). *Structured model outputs*. Retrieved October 13, 2026, from https://developers.openai.com/api/docs/guides/structured-outputs

Rastogi, A., Zang, X., Sunkara, S., Gupta, R., & Khaitan, P. (2020). Towards scalable multi-domain conversational agents: The Schema-Guided Dialogue dataset. *Proceedings of the AAAI Conference on Artificial Intelligence, 34*(5), 8689–8696. https://doi.org/10.1609/aaai.v34i05.6394

Supabase. (n.d.). *Row level security*. Retrieved October 13, 2026, from https://supabase.com/docs/guides/database/postgres/row-level-security

Wu, C.-S., Madotto, A., Hosseini-Asl, E., Xiong, C., Socher, R., & Fung, P. (2019). Transferable multi-domain state generator for task-oriented dialogue systems. In A. Korhonen, D. Traum, & L. Màrquez (Eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 808–819). Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-1078

Appendix A Run the application

Source and environment

Source: https://github.com/KarthikRamesh9149/NLP-AT2

Live application: https://formbridge-nlp-at2.vercel.app/

Use Node.js 24, pnpm 11.24.0 and Git. The commands below select the application version evaluated in this report. The report and saved results are included in later commits on the repository's main branch.

```
git clone https://github.com/KarthikRamesh9149/NLP-AT2.git
cd NLP-AT2
git checkout b0d389e410af735ba622b41a937adc351162058b
pnpm install --frozen-lockfile
cp .env.example .env.local
```

Edit `.env.local` in a text editor. Set `OPENAI_API_KEY` to a key from your own API project, with access to the configured models. Keep the key out of source control. Set `OPENAI_MODEL=gpt-6-luna` for the report's direct baseline; the template's older direct-model default is not the reported configuration. Keep `FORMBRIDGE_MODEL=gpt-6-luna` and `FORMBRIDGE_REPAIR_MODEL=gpt-6.1-sol`.

Shared features require a dedicated Supabase project. Apply every file in `supabase/migrations/` in filename order. Set the public project URL and publishable key, plus the server-only `SUPABASE_SECRET_KEY`. Enable anonymous sign-in in Supabase before setting `NEXT_PUBLIC_DEVICE_SIGN_IN_ENABLED=true`. Rebuild after changing public environment variables. Keep real email delivery disabled until a sender and the `/auth` redirect have been verified.

```
pnpm test
pnpm typecheck
pnpm build
pnpm start
```

Open `http://localhost:3000`. Keep this terminal running and use a second terminal, in the same directory, for route checks or dataset commands. Stop and restart the server after rebuilding. `pnpm dev` supports development without shared storage; production AI routes deliberately reject requests when the shared allowance database is absent.

Testing the application

The repository's `docs/OPERATIONS.md` describes workspace setup, permissions, usage limits and connector simulations. `scripts/check-restaurant-workspace.ts` creates test accounts and enquiries in a configured Supabase project. It requires the Supabase variables above. Use a dedicated test project and retain the output so the test records can be identified. Calls to OpenAI consume API usage.

The `/auth/preview` route requires no mail sender. It accepts sample `.test` addresses, keeps its account and inbox in memory and clears them on refresh. Its verification and recovery links expire after 15 minutes and can be used once. This simulation has no access to real team records.

Appendix B Reproduce the evaluation

Dataset and configuration

Run these commands from the application directory in a second terminal. Dataset acquisition is separate from installing the app. Preserve the upstream Creative Commons Attribution-ShareAlike 4.0 licence and attribution with dataset-derived material.

```
git clone \
  https://github.com/google-research-datasets/dstc8-schema-guided-dialogue.git \
  ../formbridge-sgd
git -C ../formbridge-sgd checkout e852981ae34990f4358979625854259302feaa78
pnpm data:audit ../formbridge-sgd

export OPENAI_MODEL=gpt-6-luna
export FORMBRIDGE_MODEL=gpt-6-luna
export FORMBRIDGE_REPAIR_MODEL=gpt-6.1-sol
```

The two selection files below exist at application revision `b0d389e` and contain the dialogue IDs used in this report. The script copies them into a new record before making API calls. Expected answers are used only for scoring.

```
EVAL_RESULTS=evaluation/results
pnpm exec tsx --env-file=.env.local scripts/evaluate.ts ../formbridge-sgd \
  --split=dev --sample=20 --transport=in-process \
  --replay=all-user-turns --modes=rules,model,formbridge \
  --selection="$EVAL_RESULTS/selection-product-v17-20261013-same-primary-development.json" \
  --tag=reproduce-report-development

pnpm exec tsx --env-file=.env.local scripts/evaluate.ts ../formbridge-sgd \
  --split=test --sample=20 --transport=in-process \
  --replay=all-user-turns --modes=formbridge \
  --selection="$EVAL_RESULTS/selection-product-v17-20261013-regression.json" \
  --tag=reproduce-report-regression
```

Saved outputs and scoring

The main results are saved in `sgd-dev-20-report-v18-20261013-development.json` and `sgd-test-20-report-v18-20261013-regression.json` under `evaluation/results/`. Each includes the selected dialogues, model settings, predictions, final states, usage and summary scores. Corresponding `selection-report-v18-...` and `progress-report-v18-...` files record the selection and progress of each run.

Use a new tag for each run because the script refuses to overwrite a selection or result. Retain all attempts, including failures, to keep an accurate account of the experiment.

The extraction version is `grounded-state-v18`. Its source checksum is `b9c32b2629269567a2fba24edd706bd7fdc102df7210c6721693b28f2a515424`; the runner checksum is `1b89aeedce527eaec164b0530f98d3072709009c57def1162fadedd40a6920ef`. The runner records both but rechecks only the extraction checksum before saving results. Matching uses `lib/evaluation-match.ts`; prefix replay uses `lib/evaluation-replay.ts`. Unchanged files do not guarantee identical hosted-model responses.

Appendix C Evidence and comparisons

Current evidence records

Table C1 lists the files supporting the reported results. Paths are relative to the repository, and each record identifies the code version and behaviour tested.

Table C1. Retained technical evidence.

Record	Scope
`evaluation/results/sgd-dev-20-report-v18-20261013-development.json`	Current three-method comparison: 20 development dialogues, 100 final field checks per method
`evaluation/results/sgd-test-20-report-v18-20261013-regression.json`	Current FormBridge regression: 20 inspected test dialogues
`evaluation/results/restaurant-workspace-production.json`	31 real production route and database checks at `0b25ba8`; provider actions simulated
`evaluation/results/restaurant-workspace-browser.md`	22 restaurant workflows and associated edge checks; final source `ddf8acd`
`evaluation/results/email-preview-verification.md`	Email simulation tests and deployed verification at `b0d389e`
`tests/`	Automated test inputs and assertions; 230 passing at `b0d389e`

Historical equal-model comparison

The earlier application at `32af459b6a1cd20ede9b08c2b73c20a0c385301e` compared Luna direct extraction with Luna primary and Luna repair in FormBridge. On 20 development dialogues, both reached 92/100 canonical fields; FormBridge matched 14/20 complete briefs. On a further 13 then-unselected eligible test dialogues, both contextual methods matched 62/65 fields and 12/13 complete briefs.

In that earlier configuration, FormBridge did not improve canonical field accuracy over direct extraction. The current release changes the application, prompts and repair model, so the difference between releases cannot be assigned to one component. Using the same model also does not guarantee equal numbers of calls or equal output limits.

The historical files and commands are documented in `evaluation/REPRODUCIBILITY.md`. They are reported separately from the current results.

Figure sources

Figures 1 and 3 are different crops of the same unaltered live capture, `docs/assessment/assets/FormBridge-core-workflow.jpg`. It shows the fictional enquiry used in the eight-task walkthrough. Figure 4 is the actual guest review capture retained in that asset directory. Figure 2 is drawn from the implemented request and storage paths.

Appendix D Guest review workflow

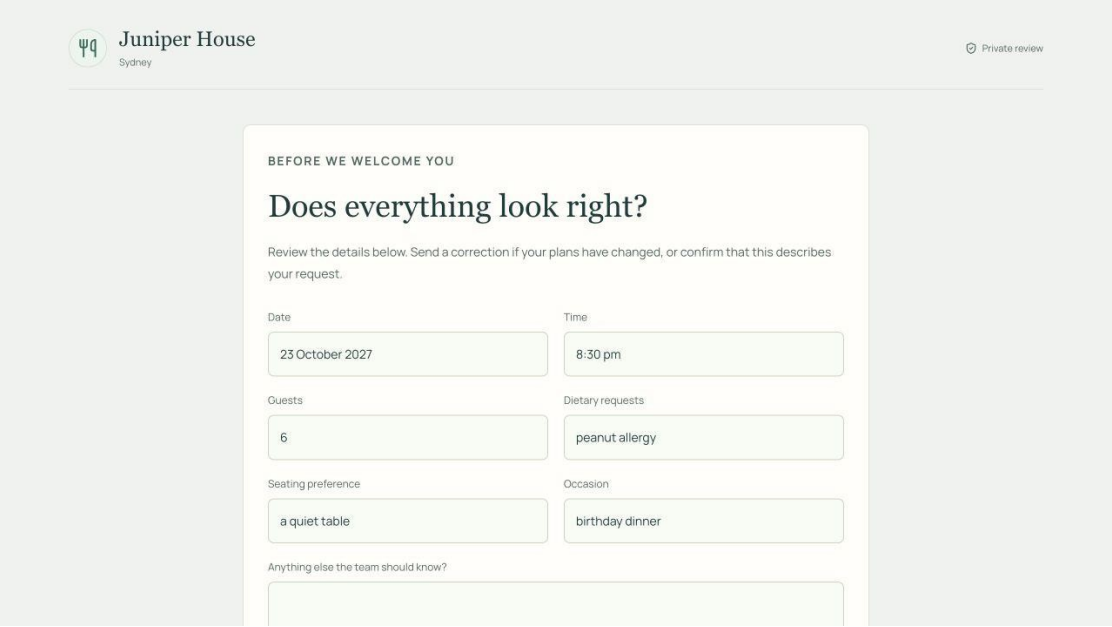


Figure 4. The deployed guest review page with a fictional enquiry. A private link presents the booking details for confirmation or correction; it does not expose the staff conversation or reserve a table.

State transitions checked in the product

Table D1. Guest review and handoff transitions.

Event	Result
Staff approve a current brief	Approval applies to that revision; a saved case can receive a private review link
Guest opens a valid link	Only the selected booking details and preferences are returned
Guest requests six places instead of five	A new customer message is queued; approval is removed and the case returns to Needs reply
Staff check the correction	The count updates while the retained date and time remain visible for review
Link is revoked or expired	The page is unavailable and reveals no saved snapshot
An unsubmitted link refers to an older case revision	The page is unavailable; staff must issue a current link
A submitted review is reopened	Its receipt remains accessible while the link and case remain valid
Simulated provider fails	The approved enquiry remains available for staff action

A guest correction must reach the correct enquiry and remove the earlier approval before staff act on it. The review workflow makes these steps visible to both the guest and the team.

Appendix E Supplementary experiments

Model-selection trials

Table E1. Historical trials on the same ten development dialogues.

Configuration	Strict fields	Complete	Failed	Mean replay
Luna primary, version 2	45/50	5/10	0/10	27.78 s
Sol primary, Astra repair, version 3	38/50	4/10	1/10	52.39 s

The second mean covers nine completed dialogues; the failed request returned HTTP 503. The files are `sgd-dev-10-grounded-luna-pilot.json` and `sgd-dev-10-grounded-v3-development.json` in `evaluation/results/`. Because prompts and validation changed between trials, the difference cannot be attributed solely to the models.

Reproduce the paired and feature checks

After the setup in Appendix A, select the revision containing the supplementary scripts. The application code is unchanged from the evaluated release. Keep the same model variables and dataset checkout. Run each command with a new tag to preserve earlier results.

```
git checkout 470c81f883065eb0a98dba7bfb7d9e05511043b5
pnpm install --frozen-lockfile
pnpm exec tsx --env-file=.env.local scripts/report-evidence.ts \
  component ../formbridge-sgd --tag=reproduce-component
pnpm exec tsx --env-file=.env.local scripts/report-evidence.ts \
  features --tag=reproduce-features
```

The paired test uses the main development selection and makes one primary call at the end of each conversation. It compares that response before and after validation, without repair or previous predictions. Failed requests remain in the denominator. The feature test uses the 20 transcripts and expected answers in `evaluation/report-feature-cases.ts`. A source check passes when a correct populated field includes a complete set of the supporting messages specified in advance. The question check tests which field is requested, and the approval check uses the application's approval function.

Files beginning `report-component-20261013-audit-followup` and `report-features-20261013-audit-followup` contain the procedures, progress and predictions in `evaluation/results/`. Both runs checked the combined source checksum before and after execution.

The eight-task live walkthrough is recorded in `evaluation/results/report-interface-walkthrough-20261013.json`, with the screenshot used for Figures 1 and 3.

Appendix F Booking desk walkthrough

The deployed booking desk was exercised with a fictional enquiry for Juniper House in Sydney on 6 November 2026. Figure 5 shows the correction history from the same capture as Figures 1 and 3.



Figure 5. The expanded history keeps the previous values visible. A revised preference removes the exact time without erasing the other booking details.

Table F1. Observed tasks in one live enquiry.

Task	Observed result
1. Open a blank desk	No sample loaded; zero of five details ready and approval disabled
2. Add a complete request	Restaurant, city, date, 7 pm and four guests shown with sources
3. Change guests and time	Six guests retained; “after 8 pm” left unresolved; approval disabled
4. Inspect sources and history	Original and correction messages visible; superseded values shown
5. Propose 8:30 pm as staff	The offer alone did not resolve the time or enable approval
6. Accept 8:30 pm as customer	Five details ready; time resolved with acceptance evidence
7. Approve the brief	Approved state and download control shown; no reservation claimed
8. Edit the earlier guest count	Count rebuilt as five; other details retained and approval removed

All eight tasks reached the expected state within this one conversation. The walkthrough tested the correction and review controls. A future participant study should measure whether users can find those controls, how long tasks take and where mistakes occur.